System Context

Generalized Notation Notation (GNN) couples a small, declarative text language for generative models in the Active Inference lineage to a pipeline of source-processing transformations and bounded execution that produces validation, visualization, simulation, and analysis artifacts (Smékal and Friedman 2023). The language gives a model one canonical written form; the pipeline gives that form many executable and graphical realizations. This section describes both halves of the architecture and the way they meet, and it states the mathematical objects the language is obliged to carry — per model kind, because the language is not tied to a single shape of generative model.

The GNN Language

A GNN model is a plain-text document organized into named sections that together pin down the generative model its author intends. The `StateSpaceBlock` declares the variables of the model and their dimensions — hidden states, observations, control factors, policies, or the system matrices of a continuous model — establishing the shape of every tensor that follows. The `Connections` section records the directed and undirected dependencies among those variables, the edges of the underlying factor graph that downstream tools read to lay out diagrams and wire up inference. Which family of generative model a file declares is decided by the notation the file carries, not by its filename or its directory: the declared matrix vocabulary is what the pipeline reads to classify the model, as sec. `sec:model_kinds` specifies. The full construct vocabulary is catalogued in

The GNN Language I

tbl. 6.

Model Kinds

Model Kinds I

Every GNN file denotes one generative model, and the language lets that model take several distinct mathematical shapes. A file whose `StateSpaceBlock` declares the categorical tensors `A`, `B`, `C`, `D` — optionally `E` — denotes a discrete state-space generative model. A file that declares the system matrices `F`, `H`, `Q`, `R` together with a Gaussian prior over the initial state (`prior_mean`, `prior_cov`) denotes a continuous linear-Gaussian state-space model. Per-agent and per-level declarations (`A_agent1`, `A_level1`, …) compose the categorical vocabulary into multi-agent and hierarchical structures. A file that declares boundary structure only — neither a categorical nor a continuous parameterization — is a structural wrapper: it names the model's interfaces but carries no renderable form.

Model Kinds I

The pipeline classifies each parsed specification structurally, and it does so from typed fields alone: the raw `## GNNSection` value, the declared matrix-key patterns, explicit agent counts, and explicit `## ModelParameters` keys. The classifier (`detect_model_kind` in `src/gnn/render/pomdp_contract.py`) resolves kinds in a fixed precedence order — multi-agent, then hybrid, then hierarchical, then continuous, then nonstationary, then learning, then factored, then structural, then the flat single-factor case. This single-winner interface identifies the most specific kind; composition-aware dispatch consults the complete `detect_model_kinds` set to admit or refuse the whole declared composition. Two properties of this classification matter for everything downstream. First, it is structural, not textual: prose in a model name or annotation never

changes how a model renders, so a passing mention of “Dirichlet” in free text cannot reroute a specification into the parameter-learning kind. Second, it is computed from the parsed specification, not asserted by the author: the same file classifies the same way on every machine, which is what lets rendering and execution behavior be stated per kind rather than per file. The taxonomy, its notation blocks, exemplar folder, and rendering and execution reach per kind are enumerated in tbl. 1. Registry flags do not establish runtime availability; execution receipts do.

Model Kinds I

Table 1: The model kinds the pipeline represents and executes, one row per kind. Notation blocks are the parameterization each kind declares; exemplar folders are the committed corpus under `input/gnn_files/`. Renderer(s)/Executor(s) cells are generated from the registry flags (see tbl. 4): the two parameterization rows carry their own flag sets, excluding continuous-only targets from categorical coverage. These flags do not establish universal multi-agent support: THRML accepts independent categorical components, and coupled models require a backend-specific representation and acceptance evidence. Native independent linear-Gaussian agent exemplars are verified on JAX and RxInfer in the v4 acceptance ledger; `recursive/` is reserved for bounded `--autonomous` proposal-loop runs and holds no committed models.

Model kind	Notation block	Exemplar folder	Renderer(s)	Executor(s)
Discrete categorical	`A/B/C/D[/E]` (column-stochastic `B` slices)	`input/gnn_files/discrete/`	PyMDP, RxInfer.jl, ActiveInference.jl, JAX, DisCoPy, PyTorch, NumPyro, Stan, bnlearn, THRML	PyMDP, RxInfer.jl, ActiveInference.jl, JAX, DisCoPy, PyTorch, NumPyro, Stan, bnlearn, THRML
Continuous linear-Gaussian	`F/H/Q/R` + priors (optional closed-loop pair)	`input/gnn_files/continuous/`	RxInfer.jl, JAX, PyTorch, NumPyro, Stan, cpomdp, ngc-learn	RxInfer.jl, JAX, PyTorch, NumPyro, Stan, cpomdp, ngc-learn
Multi-agent	`nr_agents` + per-agent categorical or linear-Gaussian parameters	`input/gnn_files/multiagent/`	Model/backend dependent composition	Model/backend dependent; explicit acceptance required
Recursive	—	`input/gnn_files/recursive/`	—	—

The Discrete Categorical Kind I

A discrete specification denotes a partially observable Markov decision process over a horizon T, factorized as in eq. 1 following the standard formulation (Da Costa et al. 2020; Smith et al. 2022).

$$P(o_{1:T},\, s_{1:T},\, \pi) \;=\; P(\pi)\, P(s_1) \prod_{t=1}^{T} P(o_t \mid s_t) \prod_{t=2}^{T} P(s_t \mid s_{t-1},\, \pi) \tag{1}$$

The four matrices named in the `StateSpaceBlock` are exactly the factors of eq. 1, and this correspondence is what makes the notation translatable rather than merely descriptive. The `A` matrix is the likelihood, a column-stochastic map from hidden states to observation outcomes, given in eq. 2.

The Discrete Categorical Kind I

$$P(o_t = i \mid s_t = j) \;=\; A_{ij}, \qquad \sum_i A_{ij} = 1 \tag{2}$$

The `B` matrix is the controlled transition dynamics: one column-stochastic slice per action, as in eq. 3. The canonical reading of the tensor is column-stochastic, `B[next_state][previous_state][action]`, matching the convention of the discrete message-passing libraries the pipeline targets (Heins et al. 2022); validation records every declared tensor's detected orientation and can transpose row-stochastic textbook literals on request, as described in sec. 4.

The Discrete Categorical Kind I

$$P(s_{t+1} = i \mid s_t = j,\ u_t = k) \ = \ B^{(k)}_{ij}, \qquad \sum_i B^{(k)}_{ij} = 1 \tag{3}$$

The `C` vector holds log-preferences over observations, which enter inference as the biased outcome distribution of eq. 4; the `D` vector is the prior over initial hidden states, eq. 5.

$$\tilde{P}(o_t = i) \ = \ \sigma(C)_i \ = \ \frac{\exp C_i}{\sum_j \exp C_j} \tag{4}$$

$$P(s_1 = i) \ = \ D_i, \qquad \sum_i D_i = 1 \tag{5}$$

Given those factors, perception is approximate Bayesian inference: a variational posterior $Q(s)$ is fitted by minimizing the variational free energy of eq. 6, which decomposes into a complexity term and an accuracy term (Friston 2010).

The Discrete Categorical Kind I

$$F[Q] = \underbrace{D_{\mathrm{KL}}[\,Q(s) \,\|\, P(s)\,]}_{\text{complexity}} - \underbrace{\mathbb{E}_{Q(s)}[\ln P(o \mid s)]}_{\text{accuracy}} \tag{6}$$

Action is expected-free-energy minimization: each policy π is scored over future steps τ by eq. 7, whose two terms are the information gain a policy is expected to yield and the extent to which its predicted outcomes match $\mathtt{C}$ (Da Costa et al. 2020).

$$G(\pi) = -\underbrace{\mathbb{E}_{Q(o_\tau,\, s_\tau \mid \pi)}[\ln Q(s_\tau \mid o_\tau, \pi) - \ln Q(s_\tau \mid \pi)]}_{\text{epistemic value}} - \underbrace{\mathbb{E}_{Q(o_\tau \mid \pi)}\left[\ln \tilde{P}(o_\tau)\right]}_{\text{pragmatic value}} \tag{7}$$

The policy posterior then combines those scores with the habit prior $\mathtt{E}$ declared in the specification, as in eq. 8, and the selected action u is sampled from it.

The Discrete Categorical Kind I

$$Q(\pi) \;=\; \sigma(\ln E - G) \tag{8}$$

The discrete categorical kind is the language's original and most densely exercised target: 29 of the 38 exemplar specifications in the corpus are discrete-state. The discrete family folder holds the kind's core set — from minimal `markov_chain.md` and `hmm_baseline.md` fixtures through epistemic planning (`tmaze_epistemic.md`) and deep planning horizons (`deep_planning_horizon.md`) to the canonical full agent (`actinf_pomdp_agent.md`) — and further categorical fixtures populate the basics, precision, structured, gridworld, and scaling-study corpora. The kind composes upward as well: multiple independent state factors (a factored model), per-level and per-agent matrix declarations (sec. `sec:structural_variants`), and Dirichlet priors that turn a declared matrix into a learned latent (sec. 7) are all extensions of this same categorical substrate.

Recent work continues to refine how expected-free-energy objectives relate to variational inference and to alternative but equivalent formulations, which is why GNN keeps the objects of eq. 1 through eq. 8 explicit in the notation rather than burying them in backend-specific code (Champion et al. 2024; Nuijten, Lukashchuk, et al. 2026; Nuijten, Laar, et al. 2026). A `ModelParameters` section fixes the scalars those objects depend on — factor cardinalities and precision terms — while a `Time` section declares whether the model is static or dynamic and, if dynamic, how the horizon T and its discretization are organized. Because every one of these sections is explicit text, a GNN file is at once human-readable, diffable under version control, and unambiguous to a parser — the property that makes the declared source contract inspectable; numerical replay also depends on the chosen

inference and runtime configuration.

The Continuous Linear-Gaussian Kind I

A file whose `StateSpaceBlock` declares `F`, `H`, `Q`, `R` together with `prior_mean` and `prior_cov` denotes a continuous-state linear-Gaussian state-space model stepped in discrete time: a Gaussian latent state $x_t \in \mathbb{R}^n$ evolves linearly, and a Gaussian observation $y_t \in \mathbb{R}^m$ reads it out linearly.

$$x_1 \sim \mathcal{N}(\mu_0,\, \Sigma_0), \qquad x_t = F\, x_{t-1} + u_{t-1} + w_t, \qquad w_t \sim \mathcal{N}(0,\, Q) \tag{9}$$

$$y_t = H\, x_t + v_t, \qquad v_t \sim \mathcal{N}(0,\, R) \tag{10}$$

Here `F` is the state-transition matrix, `H` the observation matrix, `Q` the process-noise covariance, and `R` the observation-noise covariance; `prior_mean` (μ_0) and `prior_cov` (Σ_0) fix the Gaussian prior over the initial state, so the model's randomness is declared entirely in its parameterization rather than in categorical tables. The shape contract mirrors the semantics: `prior_mean` must carry

exactly n entries, and `Q`, `R`, and `prior_cov` must be $n \times n$, $m \times m$, and $n \times n$ respectively — a mismatch is a validation-time diagnostic in the notation, not a runtime surprise inside a numerical backend.

The kind also carries an optional closed-loop control structure. When a specification declares `goal_mean` ($\mu^\star$, the preferred state) and `control_gain` (k, a scalar proportional gain) alongside a control input u_t, the rendered model closes the loop on beliefs: the controller steers the filtered posterior mean toward the preferred state.

The Continuous Linear-Gaussian Kind I

$$u_t \;=\; k\,(\mu^\star - \mu_t), \qquad \mu_t = \mathbb{E}[x_t \,|\, y_{1:t}] \tag{11}$$

This is how `input/gnn_files/continuous/continuous_navigation.md` is built: a two-dimensional navigator whose position persists between steps ($F = I$), whose observation is a noisy identity readout of that position, and whose control input pushes the believed position toward `goal_mean` with gain `control_gain`. The other continuous exemplars are passive: `predictive_coding_agent.md` and `stochastic_dynamics.md` filter without steering. In total the corpus ships 9 continuous linear-Gaussian exemplars in the continuous family folder. Symbolically the kind reuses letters the discrete vocabulary already claims — most prominently `F`, which names the state-transition matrix here and the variational free energy in eq. 6 — and sec. 9 resolves that collision by scoping each symbol table to its kind.

The remaining kinds are structural compositions of the two parameterization vocabularies above, and the classifier recognizes each by its declared key pattern rather than by convention.

Multi-agent. A specification with an explicit agent count greater than one, or with per-agent matrix keys (`A_agent1`, `B_agent1`, `C_agent1`, …), declares a multi-agent model: each agent carries its own categorical vocabulary, and coordination is expressed in the `Connections` section rather than by merging the agents into one joint table. `input/gnn_files/multiagent/stigmergic_swarm.md` is the worked exemplar — three agents coordinating through environmental traces (stigmergy) on a shared grid, with no direct communication between them — alongside `multi_agent_coordination.md` and a compact

clustered mean-field acceptance fixture.

Hierarchical. Per-level matrix keys (`A_level1`, `B_level1`, `A_level2`, ...) declare a multi-level model in which one level's beliefs supply another level's context — `input/gnn_files/hierarchical/hierarchical_pomdp.md` couples a fast lower level to a slow higher level that switches its context, and `temporal_hierarchy.md` develops the same structure over time. These fixtures explicitly declare `block_reset_v1` and `timed_soft_controller_v1`. JAX preserves their block resets and approximate 10/100-clock controller semantics; other backends report these contracts unsupported. Generic hierarchical strategies without these contracts retain their separate capability boundaries.

Learning and factored. A specification that declares a Dirichlet prior over one of its matrices (`dirichlet_A`, …) — or declares a learning `## GNNSection` value — is the parameter-learning kind: a matrix becomes a latent variable inferred from observations rather than a fixed table. `input/gnn_files/learning/dirichlet_likelihood_learning.md` is the worked exemplar: its `StateSpaceBlock` still declares `A[3,3]` in the categorical vocabulary, but `A` is a latent variable with prior pseudo-counts declared in `dirichlet_A[3,3]`, and hidden-state inference and likelihood learning run jointly by variational message passing with a mean-field factorization. The file is explicit about what its `InitialParameterization` means — the numeric `A` there is the environment's ground-truth likelihood that the agent never

observes and must recover, not the agent's belief — and its annotation records why the Dirichlet prior is identity-biased rather than uniform: a fully uniform prior leaves the column-permutation symmetry unbroken and inference converges to a label-switched optimum. A specification whose `## ModelParameters` declare more than one independent state factor without per-level or per-agent keys is the factored kind, the standard multi-factor categorical POMDP.

Structural. A file that declares boundary structure only — a Markov-blanket-style wrapper with neither a categorical `A`/`B`/`C`/`D`[/`E`] parameterization nor a continuous `F`/`H`/`Q`/`R` one — is classified structural and is informational, with no renderable model body: it names interfaces the surrounding system can reason about, and every framework reports it `unsupported` with the reason that it has no renderable form, rather than being forced through a discrete render path that would fail on missing matrices.

The recursive corpus. The `input/gnn_files/recursive/` directory is reserved for bounded `--autonomous` proposal-loop runs and holds no committed models; recursion today enters the taxonomy as structure — a level referencing its own kind, as in the hierarchical compositions above — rather than as a separately renderable kind. The corpus directory is part of the tree so that autonomous runs have a bounded, inspectable home, and its emptiness is a fact of the current release rather than an omission of the documentation.

Reading Exemplars by Kind I

Each kind is best understood through a specification that exercises it, and the corpus pairs every kind with at least one canonical exemplar. Reading one file per kind side by side makes the notation-block difference — the entire content of the taxonomy — concrete.

The discrete canonical is `input/gnn_files/discrete/actinf_pomdp_agent.md`, a fully controllable one-factor POMDP. Its `StateSpaceBlock` declares `A[3,3]`, `B[3,3,3]`, `C[3]`, `D[3]`, and `E[3]` — one observation modality, one hidden-state factor, and three actions — and every tensor of eq. 1 through eq. 8 is present by name: `A` carries the likelihood mapping, `B` carries the controlled transitions in the canonical column-stochastic slice order,

`C` holds log-preferences, `D` the initial-state prior, and `E` the habit prior. The `Connections` section then wires the inference cycle explicitly — `D>s` conditions the initial belief, `s-A` ties states to observations, `C>G` and `E>`π feed preference and habit into policy scoring, and π`>u` closes the loop from policy to action — so the file reads as a literal transcription of eq. 6 and eq. 7 into declared variables and edges.

The continuous canonical is `input/gnn_files/continuous/continuous_navigation.md` (eq. 9 through eq. 11). Its `StateSpaceBlock` declares no categorical tensors at all: `x[2,1]` and `y[2,1]` are the Gaussian state and observation, `F[2,2]`, `H[2,2]`, `Q[2,2]`, and `R[2,2]` carry the linear dynamics and noise, `prior_mean[2]` and `prior_cov[2,2]` fix the initial belief, and `goal_mean[2]` and `control_gain[1]` add the closed-loop control structure. The `InitialParameterization` gives the matrices numerically — identity dynamics and identity readout, diagonal process and observation noise, a prior centered on the origin, a preferred position, and the proportional gain — and the `Equations` section restates

eq. 9, eq. 10, and eq. 11 as the model's declared dynamics. Nothing about the file is a discretized stand-in for a POMDP; it is a linear-Gaussian model by declaration, which is what lets the classifier label it `CONTINUOUS` and the continuous-capable backends render it natively.

The composed kinds show their structure in their key names. `input/gnn_files/multiagent/stigmergic_swarm.md` declares `A_agent1[4,9]`, `B_agent1[9,9,4]`, `C_agent1[4]`, and `D_agent1[9]`, then repeats the block for each further agent, so the per-agent pattern that the classifier keys on is visible directly in the `StateSpaceBlock`; coordination between the agents is expressed through shared environment variables in `Connections`, not by merging the agents into one joint table. `input/gnn_files/hierarchical/hierarchical_pomdp.md` declares

Reading Exemplars by Kind I

`A_level1[4,4]` and `A_level2[4,2]` with matching per-level transitions and priors — the `block_reset_v1` execution contract preserves those levels and resets through JAX rather than flattening this fixture or routing it through RxInfer.jl. In every case the classification is a mechanical reading of the declared names, which is the property that keeps the taxonomy honest: a file is its kind because of what it declares, not because of where it sits.

What the Kind Classification Governs I

The classification is not an annotation a report displays and forgets; it governs observable pipeline behavior at each stage that touches a model. At the render step, the per-framework receipts group outcomes into three disjoint sets — frameworks processed, frameworks failed, and frameworks unsupported — and the unsupported set is excluded from the success denominator rather than counted against it: a categorical backend facing a continuous model, or any backend facing a structural wrapper, reports a status with a reason and drops out of the rate, so a pipeline run over a mixed-kind corpus neither inflates its failures nor hides its gaps. At the execute step the same discipline applies, with skipped and degraded lanes recorded as explicit statuses. At the analysis and reporting steps, the receipts keep the distinctions

legible: a continuous model whose categorical-only lanes report `unsupported` by capability reads differently from a discrete model whose continuous attempt failed, and the per-family acceptance ledger that the model-family gate writes carries those statuses per family, per backend, per step.

The governance runs in the direction the taxonomy promises: classification is computed from the parsed specification's declared fields, and composition-aware adapters must preserve the whole kind set. Source-bound render receipts, execution traces, and acceptance ledgers make their handling of those kinds auditable. A reader auditing any of those artifacts can recompute the classification from the file's declared vocabulary alone — tbl. 1 is the key — and expect to match what the pipeline recorded, which is what makes the kind taxonomy a checkable claim rather than a documentation convention.

The Processing Pipeline

The Processing Pipeline I

The pipeline is a fixed sequence of 25 numbered steps, 0–24, each a self-contained stage that consumes the artifacts of its predecessors and writes typed outputs for those that follow. Early steps parse and type-check the GNN text and validate it against the language schema; middle steps render visualizations, export the model to executable backends, and run simulations; later steps perform analysis, reporting, and downstream integration. The declared orchestration prerequisites among the steps form the directed acyclic graph shown in fig. 1. The graph records which steps must precede others; artifact receipts separately identify source inputs and produced files, including optional enrichments.

The Processing Pipeline I

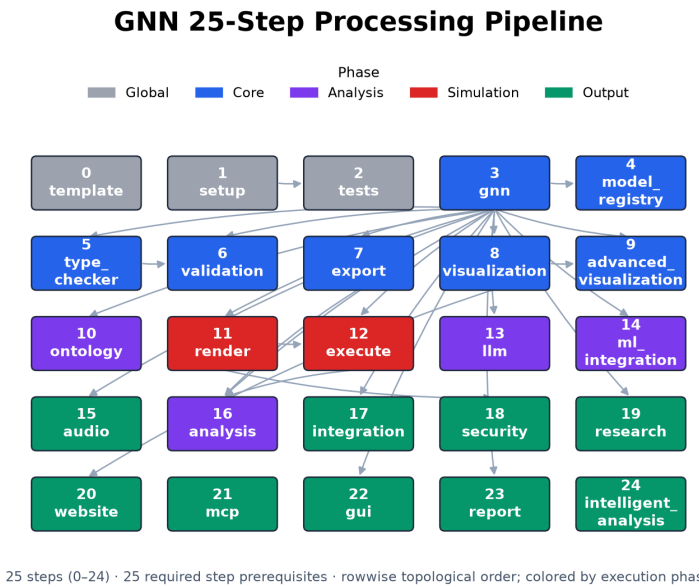


Figure 1: The 25-step GNN processing pipeline as a directed acyclic graph, arranged in rowwise topological order and colored by execution phase (legend above the nodes). Each node is the thin step orchestrator under `src/gnn/` named in tbl. 2; each arrow is a required orchestration prerequisite parsed from the Data Dependency Graph block of `src/gnn/STEP_INDEX.md`, which mirrors the live step registry. Parsing (Step 3) precedes many later steps. Optional artifact enrichments are documented separately and are not shown as required edges. The figure is generated from `src/gnn/STEP_INDEX.md` at commit 739af221e; its footer restates the step count, the number of required prerequisites, and the layout rule.

The Processing Pipeline I

The per-step responsibilities are enumerated in tbl. 2; each row names a step and the transformation it owns within the 0–24 range.

The Processing Pipeline I

Table 2: The pipeline steps with the thin orchestrator module that owns each and the one-line purpose parsed from the master table in `src/gnn/STEP_INDEX.md`. Read the Step column against the arrows of fig. 1: each row's purpose is the transformation that module owns, and the table is regenerated from the index at every build, so it cannot drift from the code it describes.

Step	Module	Purpose
0	`0_template.py`	Pipeline template & initialization
1	`1_setup.py`	Environment setup & UV dependency install
2	`2_tests.py`	Test suite execution (pytest)
3	`3_gnn.py`	GNN file discovery & multi-format parsing
4	`4_model_registry.py`	Model versioning & registry management
5	`5_type_checker.py`	GNN type validation & resource estimation
6	`6_validation.py`	Consistency & semantic quality checking
7	`7_export.py`	Multi-format export (JSON, XML, GraphML, GEXF, Pickle)
8	`8_visualization.py`	Graph & matrix visualization generation
9	`9_advanced_viz.py`	Interactive / advanced visualization (Plotly, D3)
10	`10_ontology.py`	Active Inference ontology processing & validation
11	`11_render.py`	Code generation for simulation frameworks
12	`12_execute.py`	Execute rendered simulation scripts
13	`13_llm.py`	LLM-enhanced analysis & model interpretation
14	`14_ml_integration.py`	Machine learning integration & model training
15	`15_audio.py`	Audio sonification generation (SAPF)

The Processing Pipeline II

16 `16_analysis.py`	Statistical analysis & cross-simulation aggregation
17 `17_integration.py`	System integration & cross-module coordination
18 `18_security.py`	Security validation & generated code scanning
19 `19_research.py`	Research tools & literature references
20 `20_website.py`	Static HTML website generation
21 `21_mcp.py`	Model Context Protocol processing & tool registration
22 `22_gui.py`	Interactive GNN constructor GUI
23 `23_report.py`	Comprehensive analysis report generation
24 `24_intelligent_analysis.py`	AI-powered pipeline analysis & executive reports

The Processing Pipeline I

This staged design keeps the architecture modular. The implementation is organized into 44 source packages, one cluster of responsibilities per concern, and is documented across 672 documentation files so that each step's contract, inputs, and outputs are specified independently of the others. New backends or analyses attach to the graph by declaring their dependencies rather than by editing a monolith, and deterministic step ordering makes the processing sequence reproducible. Artifact receipts bind each run to its selected model sources and resolved configuration; numerical replay also depends on the backend, dependency versions, dtype, and random seed, while run identifiers and timestamps may differ. The pipeline is kind-agnostic in its plumbing and kind-aware at its edges: parsing, validation, and export operate on whatever the file declares, while the rendering and execution steps consult the

classification of sec. `sec:model_kinds` to decide which backends a given model can reach and record, explicitly, the ones it cannot.

The Triple Play

The Triple Play I

The reason for separating a single written language from a multi-stage pipeline is the design goal GNN calls the Triple Play: one model specification, three coordinated modes of existence. The same GNN text supplies a human-readable model description, the declared state-space and factor structure for graphical visualization, and input to supported executable renderers. Source identity and validation receipts connect these representations to the authored model; runtime readiness and model-kind admission govern which simulations can execute. fig. 2 depicts these three faces and the shared specification at their center.

The Triple Play I

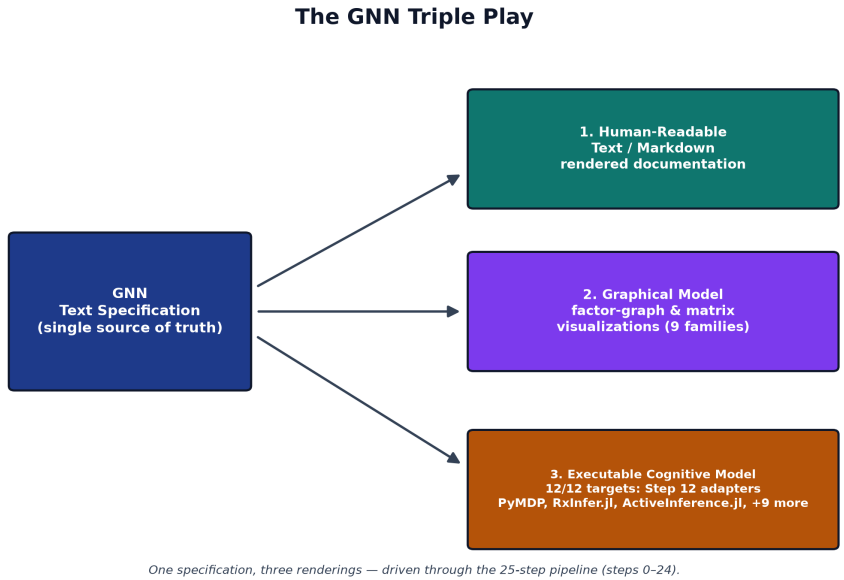


Figure 2: The Triple Play: one GNN text specification (left) and its three coordinated projections (right) — the human-readable text form, graphical model visualizations, and executable simulation code. All three arrows originate at the same source box to show their shared parsed input. Source binding, parameter validation, and semantic-fidelity checks provide evidence of agreement among the projections. The executable node summarizes 12 of 12 registered render targets with implemented Step 12 execution adapters and previews three by name; execution depends on runtime readiness, admitted model kind, and a successful source-bound run receipt. Counts are producer tokens at commit 739af221e, and the panel is generated by `scripts/manuscript_fig_triple_play.py`. Take-away: a model is authored once and then read, seen, and run from that single source.

The Triple Play I

Each face derives from a shared authored source. The text is the contract a researcher reads and reviews; the visualizations expose the declared factor-graph structure for inspection and communication; and the executable rendering targets inference frameworks, connecting the written specification to the discrete message-passing libraries for categorical Active Inference (Heins et al. 2022), to the reactive message-passing and probabilistic-programming engines that carry the linear-Gaussian kind, and to the broader tooling ecosystem for compositional model representation (Felice et al. 2021). Common source identity makes the projections traceable, while parameter values, tensor orientation, model-kind semantics, and source-bound numerical results still require validation to establish agreement. A supported model can therefore be read, seen, and run without authoring a separate specification for each purpose (Smith et al. 2022). The Triple Play is

governed per kind: renderers and executors apply the computed classification and their declared capability boundaries, and receipts distinguish successful, unsupported, skipped, partial, and failed outcomes for the selected model.